Rockchip Developer Guide PCIe EP 标准卡

文件标识：RK-KF-YF-489

发布版本：V3.5.0

日 期：2026-05-28

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福州市铜盘路软件大道89号18号楼

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档对RK PCIe 标准EP开发包使用进行说明。

产品版本

芯片型号	内核版本
RK3588	Linux 6.12、6.1、5.10、RT-Thread v5.1.x、v4.1.x
RK3568	Linux 6.12、6.1、5.10、4.19、RT-Thread v5.1.x、v4.1.x
RK1820	Linux 6.12、RT-Thread v5.1.x

说明：

- 仅在 RK PCIe EP 侧实现 RT-Thread 支持

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	yp.xiao, Kever Yang, Jon Lin	2023-03-07	初始版本
V1.1.0	yp.xiao	2023-05-22	更新SDK API以及部分配置说明
V1.2.0	yp.xiao	2023-06-15	添加EP卡固件编译、烧录、OTA升级说明 添加应用异常说明 更新`struct pcie_dev_st`结构体，支持超时配置、PCIE速率获取、RC DMA配置。
V1.3.0	Jon Lin	2023-10-07	增加pcie.bin说明
V1.4.0	yp.xiao, Jon Lin	2023-10-23	更新SDK API 添加EP验收标准章节 更新双存储说明 更新PCIe BIN说明
V1.5.0	Jon Lin	2024-05-23	更新PHY Mode设置
V1.6.0	Jon Lin	2024-09-18	添加用户态测试Demo - pcie_speed_test性能参考、增加BAR1/5支持
V1.7.0	yp.xiao, Jon Lin	2025-01-24	chips_connect开发包增加OSAL、兼容Windows版本
V1.8.0	Jon Lin	2025-02-12	优化EP启动方案说明、修改排版问题
V2.0.0	Jon Lin	2025-03-09	增加RK芯片互联独立供电方案说明
V3.0.0	Jon Lin	2025-05-14	调整须知为EP卡规格、优化EP Boot说明、增加数据传输原理说明、增加用户层多应用说明
V3.1.0	Jon Lin	2025-07-12	添加 RK1820
V3.2.0	Jon Lin	2025-08-11	独立输出标准卡文档
V3.3.0	yp.xiao	2026-03-20	完善Windows开发说明，添加win_driver目录结构和驱动架构说明，添加build/windows工程结构说明，添加版本号管理
V3.4.0	Jon Lin	2026-04-23	强调SPI Flash作为启动方案的应用场景

版本号	作者	修改日期	修改说明
V3.5.0	Jon Lin	2026-05-28	新增 DMA PRPS 链表传递支持，优化非连续内存传输性能

目录

Rockchip Developer Guide PCIe EP 标准卡

1. EP卡概述
2. EP卡规格
    - 2.1 基本规格
    - 2.2 软件开发
    - 2.3 操作系统
    - 2.4 时钟要求
    - 2.5 PCI设备ID
    - 2.6 热插拔
    - 2.7 辅助信号
    - 2.8 性能规格
    - 2.9 版本号
3. EP卡操作指南
    - 3.1 环境准备
        - 3.1.1 硬件环境
        - 3.1.2 软件环境
    - 3.2 功能验证
        - 3.2.1 Linux RC
        - 3.2.2 Windows RC
    - 3.3 实测性能
    - 3.4 Samples编译
4. EP卡业务基础
    - 4.1 原理介绍
5. Chips Connect简介
    - 5.1 原理介绍
    - 5.2 PCIe传输系统架构
    - 5.3 BUFF管理结构及传输
        - 5.3.1 初始化阶段
        - 5.3.2 发送数据
        - 5.3.3 接收数据
        - 5.3.4 BUFF配置
    - 5.4 用户层内存配置
    - 5.5 DMA PRPS 链表传递支持
    - 5.6 用户层驱动优点
    - 5.7 用户层多个独立应用扩展
6. Chips Connect配置
    - 6.1 EP卡用户层设备驱动
        - 6.1.1 Using-RC-DMA模式
7. Chips Connect典型业务场景
    - 7.1 EP视频加速卡
8. Chips Connect验收标准
    - 8.1 开发包清单
    - 8.2 补丁更新 | 一键生效"更新包"源码
    - 8.3 功能 | 关键日志信息
        - 8.3.1 RC端lspci关键信息
        - 8.3.2 EP端spl.bin启动 log
        - 8.3.3 EP端pcie.bin启动 log
    - 8.4 功能 | 固件烧录
    - 8.5 性能 | 用户态测试Demo - pcie_speed_test
        - 8.5.1 测试目标
        - 8.5.2 测试方法

8.5.3 RK3588 Gen3x4测试结果
8.5.4 RK3588 Gen2x1测试结果
8.5.5 RK1820 Gen2x1测试结果
8.6 性能 | 用户态测试Demo - pcie_loopback_test
8.6.1 测试目标
8.6.2 测试方法
8.6.3 RK3588 Gen2x1测试结果
8.6.4 RK1820 Gen2x1测试结果
8.7 兼容性 | 枚举
8.8 兼容性 | EP复位RC检测并重新枚举
8.9 功能 | OTA
9. Chips Connect异常处理
9.1 RC端x86平台执行初始化PCIE报如下错误mmap failed, Operation not permitted
9.2 HugePage配置
9.2.1 X86平台
9.2.2 RK平台
9.3 使用API接口获取user cmd区域或者ep info区域后，操作该区域导致内核报错或者报错Bus error
9.4 Windows驱动编译环境构建
9.4.1 环境搭建步骤
9.5 Windows应用执行报错：MSVCP140.dll和vcruntime140.dll找不到
9.6 Windows开启调试模式报错尝试修改调试程序设置时出错。该值受安全引导策略保护，无法进行修改或删除。
10. Chips Connect API参考

1. EP卡概述

RK PCIe EP标准卡开发包是RK"PCIe EP 卡形态产品"的软硬件综合开发包，开发基于RK EP卡Demo板（以下简称EP卡），开发包主要包含：

- EP卡的快速操作指南
- EP卡启动方案、PCIe相关软硬件设计开发以及升级OTA相关内容
- EP卡业务基础
- EP卡业务基础框架下的业务实例

2. EP卡规格

2.1 基本规格

主芯片	启动方案	SMBus扩展	独立时钟方案
RK3568	支持Flash Boot、Common Boot 不支持PCIe Boot	N	支持SRNS 不支持SRIS
RK3588	支持Flash Boot、Common Boot 不支持PCIe Boot	N	N
RK1820	支持Flash Boot、Common Boot、EP Boot	Y	N

说明：

- Flash Boot包括：
  - Flash Boot - 集成PCIe Bin及EP Boot软件方案
  - Flash Boot - 集成PCIe Bin软件方案
- 基于SPI Flash的通用启动方案（针对通用RC）
  - 性能：PCIe快速初始化（≤100ms）
  - 优势：初始化程序驻留于可编程固件中，支持灵活烧录与升级，系统兼容性强
  - 建议：针对主要RC产品做枚举、数据拷机等基础兼容性测试
- 基于PCIe Boot的专用启动方案（针对专用RC平台，且需要EP芯片支持PCIe Boot）
  - 性能：PCIe极速初始化（≤20ms），适配严苛的启动时序要求
  - 成本：精简硬件架构，省去一颗外部SPI NOR Flash物料
  - 注意：采用固化初始化逻辑，该固件不支持二次烧录与调试

2.2 软件开发

EP卡开发包本质为PCIe产品功能展示开发包，非一站式解决方案，需客户基于此做PCIe及上层应用的二次开发。

2.3 操作系统

EP设备：

- EP卡兼容Linux、Android、RT-Thread操作系统

RC设备：

- EP卡兼容Linux操作系统的RC设备，已在特定RC上完成验证
- EP卡兼容Windows操作系统的RC设备，已在特定RC上完成验证，但仅实现基础传输功能，其余功能需客户自行参考Linux驱动进行开发

2.4 时钟要求

EP卡推荐使用同源参考时钟方案，即EP使用RC端提供的100M参考时钟。

对于RC和EP均采用RK芯片、且支持独立时钟方案的EP卡，RC&EP采用内部100M时钟，可以省掉外置的时钟发生器和时钟buffer芯片。

2.5 PCI设备ID

RK PCIe EP卡默认ID为RK自定义设备ID，对于第三方开发用户，要求：

- **内部使用设备**：如果设备仅在封闭环境中使用，可以使用RK PCIe已有的VID
- **对外设备**：如果设备是对外销售或公开使用的，需要自行向PCI-SIG申请VID

2.6 热插拔

RK PCIe EP卡不支持热插拔。

2.7 辅助信号

辅助信号	功能	标准EP方案	EP芯片独立供电方案
PRSNT#	N		
PERST#	GPIO输入	必选，接入系统复位，整板复位	必选，GPIO输入，软件复位，配置dts `reset-gpios`
CLKREQ#	FUNCTION输入	可选，用以扩展ASPM L1SS功耗管理，暂未实现但建议预留，如明确无需求可不接	可选，用以扩展ASPM L1SS功耗管理，暂未实现但建议预留，如明确无需求可不接
JTAG	N		
SMBus	N		
WAKE#	GPIO输出	必选，扩展休眠唤醒功能，暂未实现但建议预留	必选，扩展休眠唤醒功能，配置dts `wake-gpios`
+3.3Vaux	N		

说明：

- EP芯片独立供电方案由于涉及单端掉电的情景，建议做IO保护，防止一端下电、另一端IO持续输出导致电压/电流灌的风险
- 功能为GPIO的引脚，请勿在pinctrl里面配置为pcie function，支持任意引脚，无限制
- 功能为FUNCTION的辅助信号，需选择对应功能脚
- 标准EP方案PERST#接入系统复位进行整板复位的需求：
  - PERST#信号作为RC端物理复位EP的最后手段
  - 存在特殊RC兼容性问题，例如：RC枚举完成后立刻进入L2并通过PERST# 复位，需快速复位
  - NOTE: 由于RC的PERST是PP强驱，EP上PMIC是OD脚，所以不管RC的PERST是3.3V还是1.8V，都需要通过电路转换来驱动EP端的RESET_L

2.8 性能规格

用户应明确PCIe传输带宽需求、数据流向（RC/EP单向？双向？）、是否有零碎控制流/大块数据流，建议：

- 阅读“实测性能”章节，了解chips_connect开发包传输上限性能
- 阅读“性能 | 用户态测试Demo - pcie_speed_test”章节，了解不同数据块大小传输带宽

2.9 版本号

RC/EP 各维护一个版本号，分别对应 Driver/Device，版本号分为 Major/Minor/Patch 各 8 bit，其中：

- Major (高 8 位)：代表破坏性更新（接口定义变了、寄存器偏移换了）
  - 校验逻辑：`Driver_Major == Device_Major`。如果不相等，驱动必须拒绝加载，否则极大概率导致系统崩溃（Kernel Panic）。
- Minor (中 8 位)：代表功能增强或修复（新增了寄存器、优化了性能）

    - 校验逻辑：`Driver_Minor >= Device_Minor`；
    - 如果驱动比设备新，驱动可以启用新功能；如果驱动比设备旧，驱动应降级运行，只使用旧功能；
- Patch (低 8 位)：通常 Patch 位的变化不影响驱动逻辑，仅用于日志记录。

3. EP卡操作指南

3.1 环境准备

EP卡支持对接通用RC以及RK RC，这里对两种场景下基础功能验证做简单说明。

3.1.1 硬件环境

RK3588 EP卡（标准卡）对接RK3588 RC参考硬件环境

- RC：RK3588-EVB1-V10
    - 连接电源线、串口线、USB转type-c固件下载线、HDMI输出线 (连接HDMI_OUT0)
- EP：RK3588-EVB4-V11
    - RC直接供电，不需要单独电源线，不支持热拔插，需要先接上再对RC上电。

RK3588 EP卡（标准卡）对接通用RC

- RC：PC
- EP：RK3588-EVB4-V11
    - RC直接供电，不需要单独电源线，不支持热拔插，需要先接上再对RC上电。

RK3588芯片互联独立供电方案参考硬件环境

- RC：RK3588-EVB1-V10
    - 连接电源线、串口线、USB转type-c固件下载线、HDMI输出线 (连接HDMI_OUT0)
- EP：RK3588-EVB4-V12
    - EP卡独立供电，R8114和R2005不贴
    - 要求接入PERST#、WAKE#信号，且作为GPIO使用
    - 连接电源线、串口线、USB转type-c固件下载线

3.1.2 软件环境

芯片需要的loader、uboot、boot、以及文件系统可在发布包的 images 目录下获得这些镜像文件。

Flash Boot - 集成PCIe Bin软件方案

以RK3588 EP卡为例，启动文件如下表所示：

项目	文件名称	描述
RC(对接通用RC不需要)	rk3588-update-rc-evb7-v11.img	烧写到RC Flash
EP	rk3588-update-ep-evb4-v11.img	烧写到EP Flash

说明：

- update.img包含了MiniLoaderAll.bin、uboot.img、boot.img、rootfs.img等文件

Flash Boot - 集成PCIe Bin及EP Boot软件方案

以RK3588 EP卡为例，启动文件如下表所示：

项目	文件名称	描述
RC(对接通用RC不需要)	rk3588-update-rc-evb7-v11.img	烧写到RC Flash
EP	RK3588_MiniLoaderAll.bin	烧写到EP Flash
EP	pcie_idb.img	烧写到EP Flash
EP	normal_idb.img	烧写到EP Flash
EP	uboot.img	由RC通过PCIE导入EP DDR 内存
EP	boot.img （包含内核+rootfs）	由RC通过PCIE导入EP DDR 内存

说明：

- RC通过PCIE对EP下载固件请参考下一章节功能验证说明

3.2 功能验证

验证PCIE EP功能的样例代码位于 SDK 发布包的examples目录下，包括EP和RC分别使用的Sample代码、封装的消息通讯代码和封装的数据通讯代码。

3.2.1 Linux RC

通用RC功能验证需要的bin和资源存放在images目录下需要手动拷贝到RC平台（仅X86平台），RK RC的资源直接打包到固件root目录中不需要额外操作。

在硬件、软件环境准备好后，执行完整的 PCIe 功能验证的步骤如下。不需要对EP卡下载固件可以直接跳过第一点。

1. RC通过PCIE对EP下载固件

   步骤1. EP卡串口波特率是1500000（RC波特率也是1500000），连接串口后会一直打印如下log，说明EP和RC的PCIE通路Link成功，正在等待RC传输固件。

```
U-Boot SPL 2017.09-g614fe864bf-dirty #xyp (Apr 20 2023 - 11:19:22)
RKEP: 191277 - Link ready! Waiting RC to download Firmware:
RKEP: 191723 - Download uboot.img  to BAR2+0
RKEP: 192068 - Download boot.img   to BAR2+0x400000
RKEP: 192466 - Send CMD_LOADER_RUN to BAR0+0x400
RKEP: 1092850 - Waiting for FW, CMD: 0
RKEP: 2093146 - Waiting for FW, CMD: 0
RKEP: 3093443 - Waiting for FW, CMD: 0
RKEP: 4093740 - Waiting for FW, CMD: 0
RKEP: 5094036 - Waiting for FW, CMD: 0
```

说明： EP卡直接进到系统说明flash中是完整固件，请进入maskrom下重新烧写`pcie_idb.img`和`normal_idb.img`。

步骤2.RC端执行`./pcie_download_test_rc 0 ./uboot.img ./boot.img`，启动RC与EP之间的数据传输，通过DMA把固件搬运到EP卡指定内存地址，然后发送启动指令让EP通过DDR启动，EP卡此时会打印正常的开机log进入系统。 说明： RC端执行应用后异常报错请查看`examples/pcie_download_test/README.md`排查问题。

2. 视频解码预览Sample

步骤1. EP串口波特率是1500000（RC波特率也是1500000），连接串口后，RC执行`lspci -vvv |grep 356a`以及`ls /dev/pcie-rkep*`可以看到如下设备说明识别到EP卡并且RC驱动加载正常。

```
[root@RK3588:/userdata]# lspci -vvv |grep 356a
01:00.0 Processing accelerators: Rockchip Electronics Co., Ltd Device 356a (rev 01)
[root@RK3588:/userdata]# ls /dev/pcie-rkep*
/dev/pcie-rkep0000:01:00.0
```

步骤2. EP卡正常进入系统后，此时终端上将打印如下log，并被阻塞，等待RC应用启动。

```
00:00:05.330 I [cmp_pcie_ep: cc_cmp_pcie_device: 390] rk pcie version: v3.0.0 - 2024-
12-11
00:00:05.330 D [chipscon: cc_kernel_api_prob: 398] open rkep: /dev/pcie_ep Success.
00:00:05.330 D [chipscon: cc_kernel_api_prob: 403] probe pcie_rkep, fd = 3
00:00:05.330 I [core_pcie_ep: cc_core_pcie_dev_i: 53] bar0 size=0x400000
00:00:05.330 I [core_pcie_ep: cc_core_pcie_dev_i: 53] bar2 size=0x4000000
00:00:05.330 I [core_pcie_ep: cc_core_pcie_dev_i: 53] bar4 size=0x100000
00:00:05.330 E [chipscon: cc_kernel_api_reso: 141] cc_kernel_api_resource_mmap mmap
failed, size=0, err=Invalid argument
00:00:05.330 E [core_pcie_ep: cc_core_pcie_dev_i: 73] no bar1
00:00:05.330 E [chipscon: cc_kernel_api_reso: 141] cc_kernel_api_resource_mmap mmap
failed, size=0, err=Invalid argument
00:00:05.330 E [core_pcie_ep: cc_core_pcie_dev_i: 86] no bar5
00:00:05.330 I [pcie_dma: cc_core_pcie_dma_i: 590]  - id=ffffffff
00:00:05.330 I [pcie_dma: cc_core_pcie_dma_i: 600]  - id=356a1d87
00:00:05.330 I [pcie_dma: cc_core_pcie_dma_i: 609]  - magic=524b4550, ver=1
......
00:00:05.331 D [cmp_pcie_ep: pcie_task_ep_msg_t: 264]  start
00:00:05.331 D [cmp_pcie_ep: stream_speed_threa: 352] stream_speed_thread start
```

步骤3. 参考用户层内存配置章节对RC系统进行内存配置。

步骤4. RC执行如下指令，完成与EP卡的握手然后启动与EP之间的数据传输。

- 通用RC：`./pcie_video_test_rc ./output1.h264 36 704 576 0 1`
- RK RC：`pcie_video_test_rc /root/output1.h264 36 704 576 0 1`

此时RC的显示设备会输出1080P60分辨率，可以看到36路视频解码以及首路NN人型检测功能。 说明： RC端是通过DRM显示框架来输出显示数据，RK RC默认支持只需要把HDMI OUT0接上显示设备即可，通用RC需要安装`libdrm-dev`组件并且通过Ctrl+F3切换到其它无GUI终端再执行测试指令。

步骤5. 如果需要停止Sample程序，可以在RC应用输入回车键，RC会发消息给EP卡销毁相关 业务，并退出程序；EP卡接收消息后销毁相关业务并退出程序。需要正常退出业务流程，否则EP卡业务不会退出导致下次运行异常。

说明： EP端应用默认开机自启并且循环启动，若需要运行其它测试程序需要先在开机脚本把当前测试注释然后重启RC和EP。

修改/etc/init.d/S99_pcie_ep.sh，把下面的代码注释。

```
while true;
do
    cd /
    pcie_video_test_ep 1
done &
```

3.2.2 Windows RC

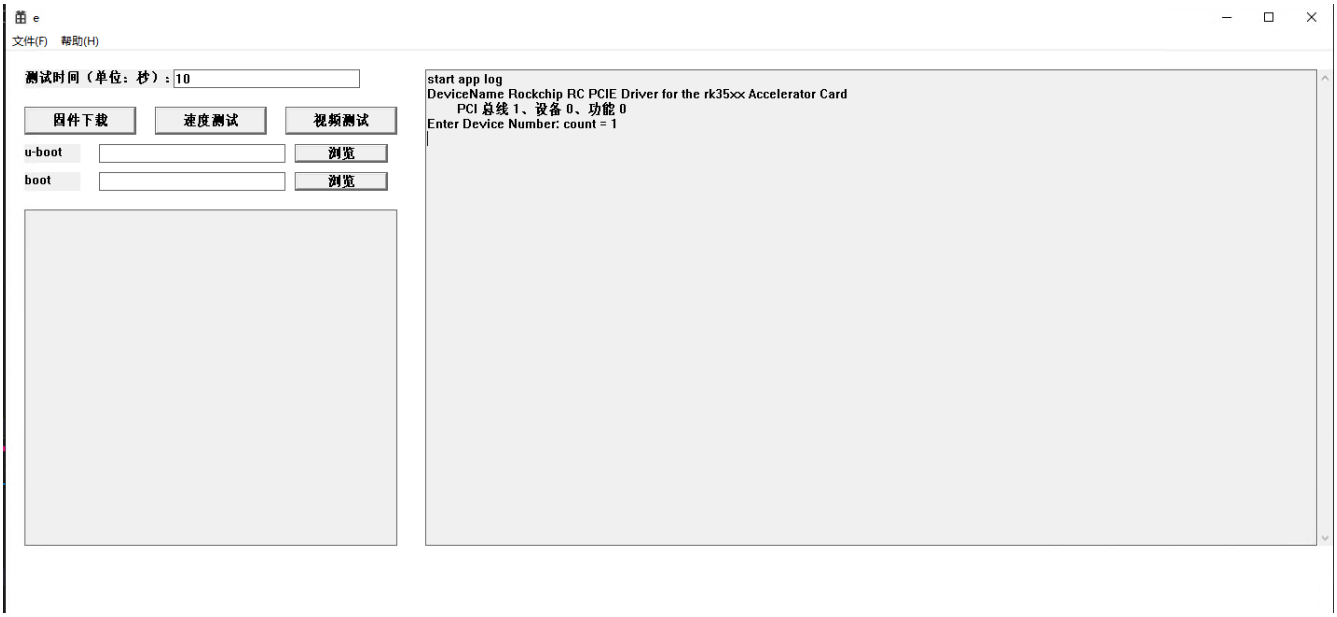
Windows RC提供的驱动基于win10 2004版本的系统开发，高版本系统需要构建驱动编译环境自行编译。

参考 Windows驱动编译环境构建 章节搭建测试环境，测试只需要关注系统版本以及禁用驱动程序强制签名步骤。

系统环境确认后，插入EP卡然后对PC上电，可以在设备管理器中看到PCI设备，右键点击该设备选择"更新驱动程序"->"浏览我的电脑以查找驱动程序"->设置为提供的驱动目录(Win-Driver-RC) ->下一步安装驱动。

安装完成后可以在设备管理器看到`Rockchip RC PCIe Driver for the rk35xx Accelerator Card`设备。

使用**管理员权限**运行测试工具，工具如下图所示，当前支持固件下载、速率测试、视频解码目标检测测试。运行工具报错可查看 `examples/pcie_download_test/README.md` 排查问题。



在硬件、软件环境准备好后，执行完整的 PCIe 功能验证的步骤如下。不需要对EP卡下载固件可以直接跳过第一点。

1. RC通过PCIE对EP下载固件

   步骤1. EP卡串口波特率是1500000（RC波特率也是1500000），连接串口后会一直打印如下log，说明EP和RC的PCIE通路Link成功，正在等待RC传输固件。

```
U-Boot SPL 2017.09-g614fe864bf-dirty #xyp (Apr 20 2023 - 11:19:22)
RKEP: 191277 - Link ready! Waiting RC to download Firmware:
RKEP: 191723 - Download uboot.img  to BAR2+0
RKEP: 192068 - Download boot.img   to BAR2+0x400000
RKEP: 192466 - Send CMD_LOADER_RUN to BAR0+0x400
RKEP: 1092850 - Waiting for FW, CMD: 0
RKEP: 2093146 - Waiting for FW, CMD: 0
RKEP: 3093443 - Waiting for FW, CMD: 0
RKEP: 4093740 - Waiting for FW, CMD: 0
RKEP: 5094036 - Waiting for FW, CMD: 0
```

   说明： EP卡直接进到系统说明flash中是完整固件，请进入maskrom下重新烧写`pcie_idb.img`和`normal_idb.img`。

   步骤2. 打开工具加载uboot和boot固件，然后点击固件下载，工具通过DMA把固件搬运到EP卡指定内存地址，然后发送启动指令让EP通过DDR启动，EP卡此时会打印正常的开机log进入系统。

2. 视频解码预览测试

   步骤1：EP卡正常进入系统后，此时终端上将打印如下log，并被阻塞，等待RC应用启动。

```
00:00:05.330 I [cmp_pcie_ep: cc_cmp_pcie_device: 390] rk pcie version: v3.0.0 - 2024-
12-11
00:00:05.330 D [chipscon: cc_kernel_api_prob: 398] open rkep: /dev/pcie_ep Success.
00:00:05.330 D [chipscon: cc_kernel_api_prob: 403] probe pcie_rkep, fd = 3
00:00:05.330 I [core_pcie_ep: cc_core_pcie_dev_i: 53] bar0 size=0x400000
00:00:05.330 I [core_pcie_ep: cc_core_pcie_dev_i: 53] bar2 size=0x4000000
00:00:05.330 I [core_pcie_ep: cc_core_pcie_dev_i: 53] bar4 size=0x100000
00:00:05.330 E [chipscon: cc_kernel_api_reso: 141] cc_kernel_api_resource_mmap mmap
failed, size=0, err=Invalid argument
```

```
00:00:05.330 E [core_pcie_ep: cc_core_pcie_dev_i: 73] no bar1
00:00:05.330 E [chipscon: cc_kernel_api_reso: 141] cc_kernel_api_resource_mmap mmap
failed, size=0, err=Invalid argument
00:00:05.330 E [core_pcie_ep: cc_core_pcie_dev_i: 86] no bar5
00:00:05.330 I [pcie_dma: cc_core_pcie_dma_i: 590]  - id=ffffffff
00:00:05.330 I [pcie_dma: cc_core_pcie_dma_i: 600]  - id=356a1d87
00:00:05.330 I [pcie_dma: cc_core_pcie_dma_i: 609]  - magic=524b4550, ver=1
......
00:00:05.331 D [cmp_pcie_ep: pcie_task_ep_msg_t: 264]  start
00:00:05.331 D [cmp_pcie_ep: stream_speed_threa: 352] stream_speed_thread start
```

步骤2：把测试片源拷贝到工具目录下，重命名为`output1.h264`，当前工具固定测试片源为h264裸码流。设置测试时间，然后点击视频测试按钮，即可运行测试。

测试画面如下：



3.3 实测性能

PCIe EP传输性能

EP	RC	硬件配置	场景	最大传输速率
RK3588	RK3588	PCIe 3.0x4Lane	EP写数据到RC	2.95GB/s
			EP从RC读数据	1.8GB/s
			EP同时读写RC	各自1.6GB/s
RK3588	RK3588 (开启Using RC DMA)	PCIe 3.0x4Lane	EP写数据到RC	2.95GB/s
			EP从RC读数据	2.95GB/s
			EP同时读写RC	各自2.85GB/s
	X86	PCIe 3.0x4Lane	EP写数据到RC	2.9GB/s
			EP从RC读数据	2.5GB/s
			EP同时读写RC	各自2.4GB/s
	Windows	PCIe 3.0x4Lane	EP写数据到RC	650MB/s
			EP从RC读数据	700MB/s
			EP同时读写RC	各自320MB/s

后续性能会继续优化，Windows暂不支持链式DMA测试，速率受内存结构影响比较大，大块连续内存速率就高

3.4 Samples编译

各sample编译以及使用说明请参看`examples/*/README`。

4. EP卡业务基础

4.1 原理介绍

EP卡业务内容主要是将源数据和处理器处理后的目标数据通过PCIe链路完成上下游间的传输，主要资源包括：

RC端

- 通过开源标准库libpci操作PCIE EP设备。
- 通过pcie-rkep驱动完成：
  - BAR映射，推荐作为控制流传输方式
  - DMA传输，推荐作为数据块传输方式
  - 处理MSI中断，用户层主动poll等待以提高响应速率，但涉及多层握手，实时性一般
  - 触发ELBI中断
  - 申请DMA内存

- 支持多进程，例如支持运行多个pcie_speed_test例程

EP端

- 通过pcie_ep驱动完成：
  - 默认 class id 0x1200（Processing accelerators）
  - BAR inbound物理地址空间映射
  - DMA传输
  - 发送MSI中断
  - 处理ELBI中断，用户层主动poll等待以提高响应速率，但涉及多层握手，实时性一般
  - 支持多线程
- VDEC、VENC、RGA、GPU、NPU等硬件资源可以通过Rockit 或者RKNN在用户层调用。

5. Chips Connect简介

5.1 原理介绍

Chips Connect开发包是基于EP卡业务基础构建的用户层开发包，主要提供RC、EP上下行数据传输、消息通知等常用功能的接口参考，同时构建典型业务场景以供参考。

BAR空间分配（common platforms)

对于内存分配较为充足的通用方案，SPL 及 Kernel 阶段BAR空间分配：

BAR空间	起始地址	结束地址	功能
BAR0	0	4KB	EP保留空间，驱动版本信息、中断状态等等。
	4KB	12KB	保留空间。
	12KB	16KB	DEBUG功能相关配置信息。
	16KB	128KB	EP设备状态信息。
	128KB	1MB	用户自定义。
	1MB	4MB	框架命令交互以及buff管理。
BAR1（Default Disabled）	0	1MB	用户自定义，如何使能参考“BAR大小配置”章节配置。
BAR2（BAR2/3 Bar Repair）	0	64MB	Using RC DMA使用。
BAR4	0	1MB	wired寄存器区，用于操作DMA。
BAR5（Default Disabled）	0	1MB	用户自定义，如何使能参考“BAR大小配置”章节配置。

说明：

- EP端BAR用户自定义空间可用于二次开发扩展，其余BAR空间为该业务代码需求空间，例如：
  - BAR1 默认关闭，全部空间为用户自定义空间，开启后用作控制流
  - BAR5 默认关闭，全部空间为用户自定义空间，开启后用作控制流
- EP端BAR inbound CPU address默认由“BAR映射配置”固定配置，支持RC端动态调整

BAR空间分配（memory-constrained platforms)

对于内存分配较为局促的特殊方案，SPL 及 Kernel 阶段BAR空间分配：

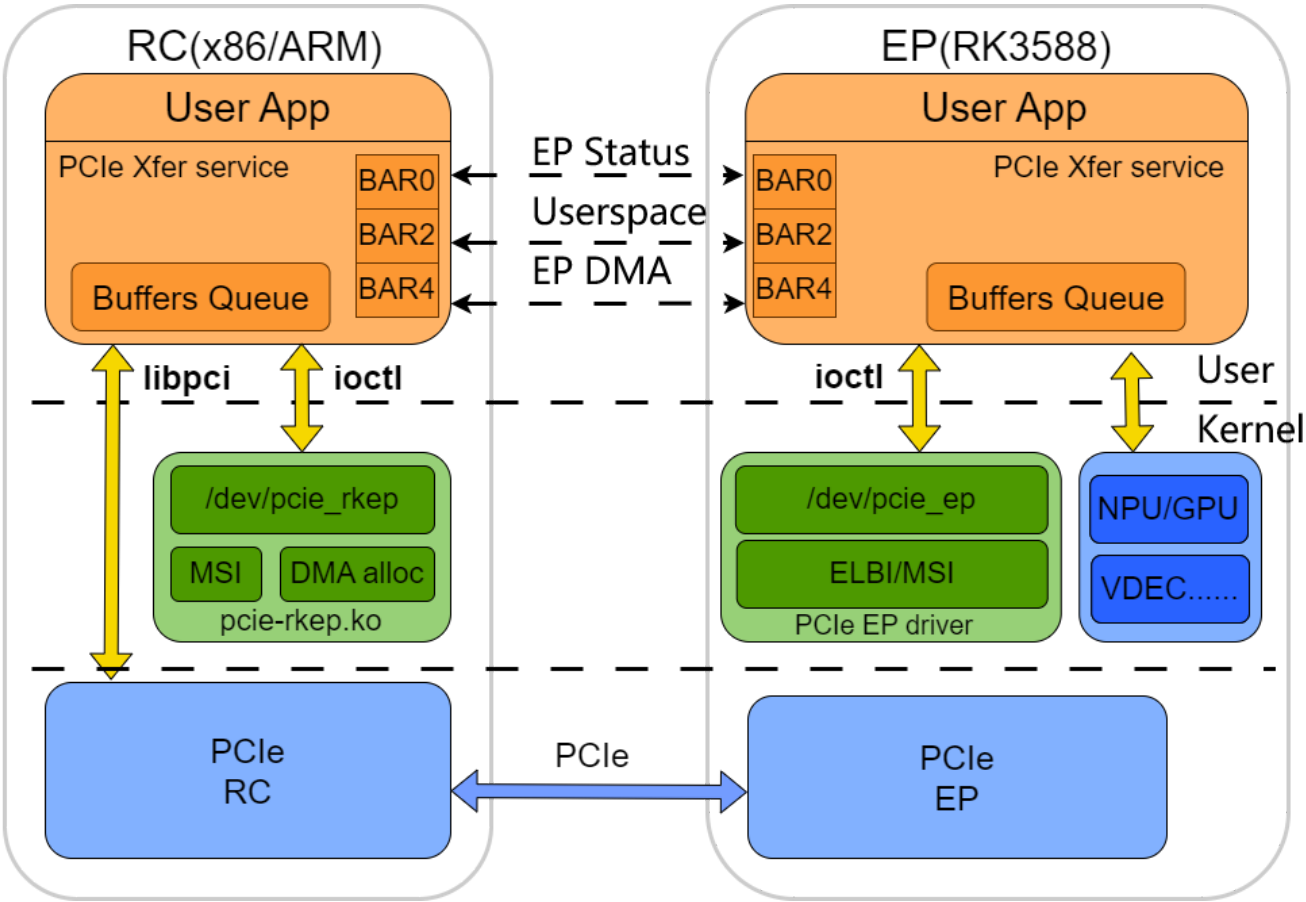
BAR空间	起始地址	结束地址	功能
BAR0	0	Reserved	预留空间，默认为0
	R	R+2048B	EP保留空间，驱动版本信息、中断状态等等。
	R+2048B	R+2560B	DEBUG功能相关配置信息。
	R+2560B	R+3072B	EP设备状态信息。
	R+3072B	R+4KB	用户自定义。
	R+4KB	R+64KB	框架命令交互以及buff管理。
BAR1（Default Disabled）		1MB	用户自定义，如何使能参考“BAR大小配置”章节配置。
BAR2（BAR2/3 Bar Repair）	0	64MB	Using RC DMA使用。
BAR4	0	4MB	wired寄存器区，用于操作DMA。
BAR5（Default Disabled）	0	1MB	用户自定义，如何使能参考“BAR大小配置”章节配置。

说明：

- 预留空间：BAR最小对齐1MB，通常要求申请1MB内存用以映射，但对于内存紧张的环境，允许做预留处理，chips connect业务使用预留后的内存
- EP端BAR用户自定义空间可用于二次开发扩展，其余BAR空间为该业务代码需求空间，例如：
  - BAR1 默认关闭，全部空间为用户自定义空间，开启后用作控制流
  - BAR5 默认关闭，全部空间为用户自定义空间，开启后用作控制流
- EP端BAR inbound CPU address默认由“BAR映射配置”固定配置，支持RC端动态调整
- "R" 即 Reserved size，部分平台BAR0并非全空间开发给chips_connect，支持预留索引到目标空间

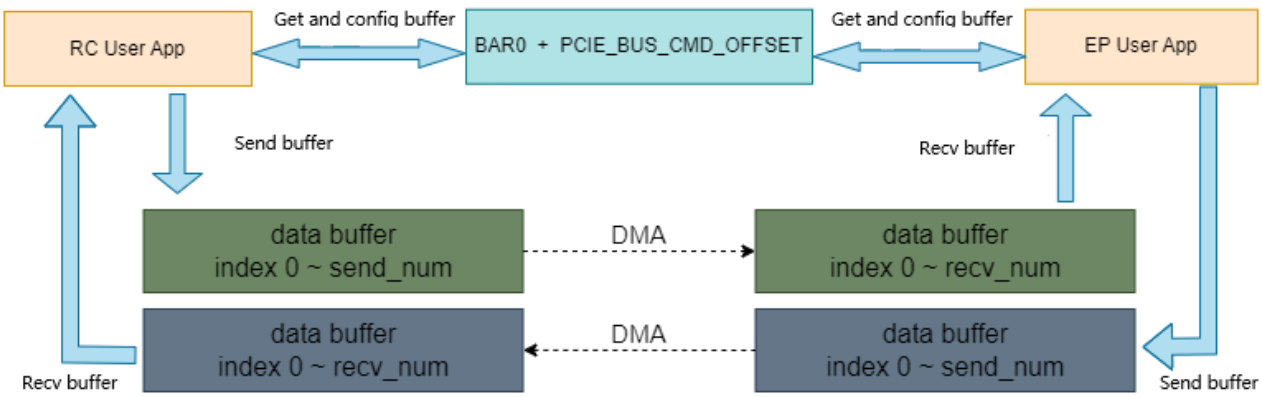
5.2 PCIe传输系统架构

PCIe RC与EP系统框图



5.3 BUFF管理结构及传输

BUFF管理



这里从RC的角度简单说明下从初始化到一次完整的读写流程：

5.3.1 初始化阶段

主要是做两件事：

1. 根据VENDOR_ID、DEVICE_ID以及总线地址初始化对应的PCIE设备。

   内部主要是通过libpci接口遍历系统PCIE设备并映射BAR空间到应用层。

支持传递不同的设备ID重复调用`rk_pcie_device_init`接口初始化设备。

2. 调用`rk_pcie_task_create`创建任务对内存、BUFF状态进行初始化，可通过配置`struct pcie_task_attr_st`结构体来设置BUFF的数量和大小，配置信息会通过BAR0空间传递给EP设备，并且等待EP设备初始化完成。EP通过`rk_pcie_get_task_msg`获取创建任务消息然后调用task初始化，两边会进行初始化配置交互。

   RC端应用根据编译配置信息，选择Hugepage/RKEP/RKDRM方式申请内存。每个EP卡设备都会在RC端生成对应的`/dev/pcie-rkep*`节点，并且申请单独的连续大内存（需要打开对应配置）。

   EP端收到配置信息后，初始化BUFF状态，并调用接口申请物理连续的内存，支持Hugepage/RKDRM方式申请内存。

   RC和EP的BUFF信息最终会赋值到BAR0特定区域，这块区域用来对BUFF状态管理。

```
#define VENDOR_ID                   0x1d87
#define DEVICE_ID                   0x356a
struct pcie_dev_bus dev_bus;
int ret;
//获取系统匹配的PCI设备总线地址
ret = rk_pcie_get_pci_bus_addresses(VENDOR_ID, DEVICE_ID, &rc_ctx.dev_bus);
if (ret != 0) {
    printf("get pci bus address fail\n");
    return -1;
}
printf("ep dev max num : %d\n", rc_ctx.dev_bus.dev_max_num);
if (rc_ctx.dev_bus.dev_max_num == 0) {
    printf("no ep device, exit ... \n");
} else {
    rc_ctx.dev_attr.using_rc_dma = 0;
    rc_ctx.dev_attr.rc_dma_chn = 0;
    rc_ctx.dev_attr.enable_speed = 1;
    rc_ctx.dev_attr.vendor_id = VENDOR_ID;
    rc_ctx.dev_attr.device_id = DEVICE_ID;
    rc_ctx.dev_attr.use_dma_only_in_rc = 0;
    rc_ctx.dev_attr.mem_type = E_MEM_TYPE_HUGEPAGE;
    memcpy(rc_ctx.dev_attr.pci_bus_address, pci_bus_address, strlen(pci_bus_address));
    //选择第一个匹配设备初始化
    rc_ctx.rk_handle = rk_pcie_device_init(&rc_ctx.dev_attr);
    //获取EP设备模式
    rk_pcie_get_ep_mode(rc_ctx.rk_handle, &rc_ctx.devmode);
    printf("ep device init, dev mode: %d, submode: %d\n", rc_ctx.devmode.mode,
rc_ctx.devmode.submode);
}
```

5.3.2 发送数据

发送数据可以分为3个步骤

1. 查询是否有可用BUFF。
2. 获取可用BUFF，并写入数据。
3. 发送BUFF到对端。

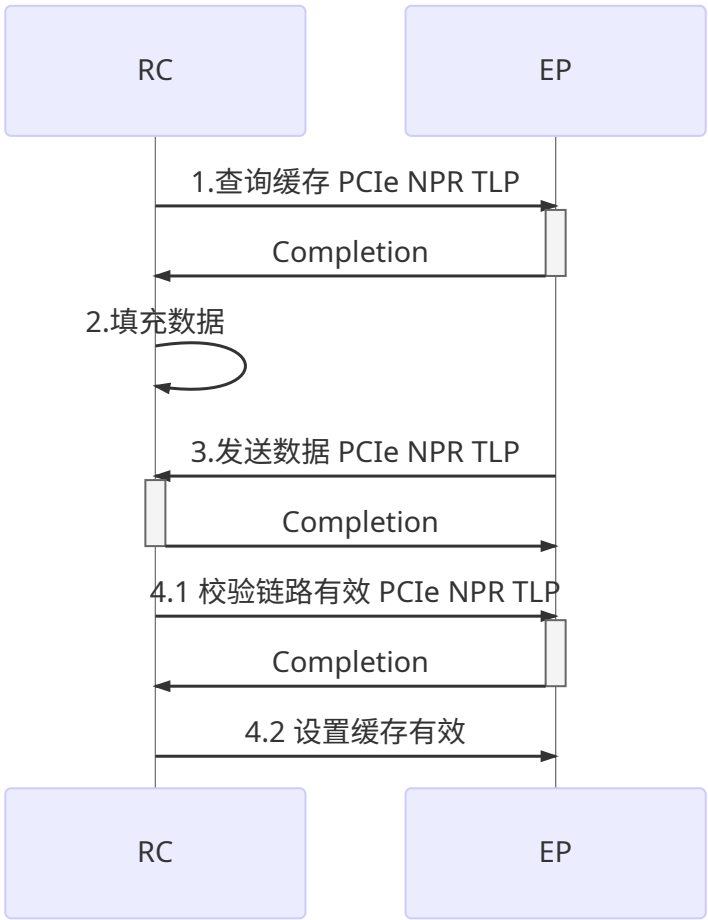
以发送BUFF举例，初始化配置`send_buff_num = 5`说明系统中最多只有5个发送BUFF，发送端调用接口获取可用BUFF后，可以通过`struct pcie_buff_node`的属性获取BUFF物理地址和虚拟地址以及BUFF大小，然后进行业务操作。最后调用发送接口把BUFF数据传输到对端，发送接口内部会先对BUFF做flush cache，保证物理内存是最新的数据，然后调用DMA接口搬运数据到对端。DMA传输的数据大小就是`node->size`，不能超过BUFF的大小。

注意：发送到对端的BUFF，需要对端释放才会重新变成可用BUFF，对端一直不释放整个流程就会被阻塞住。

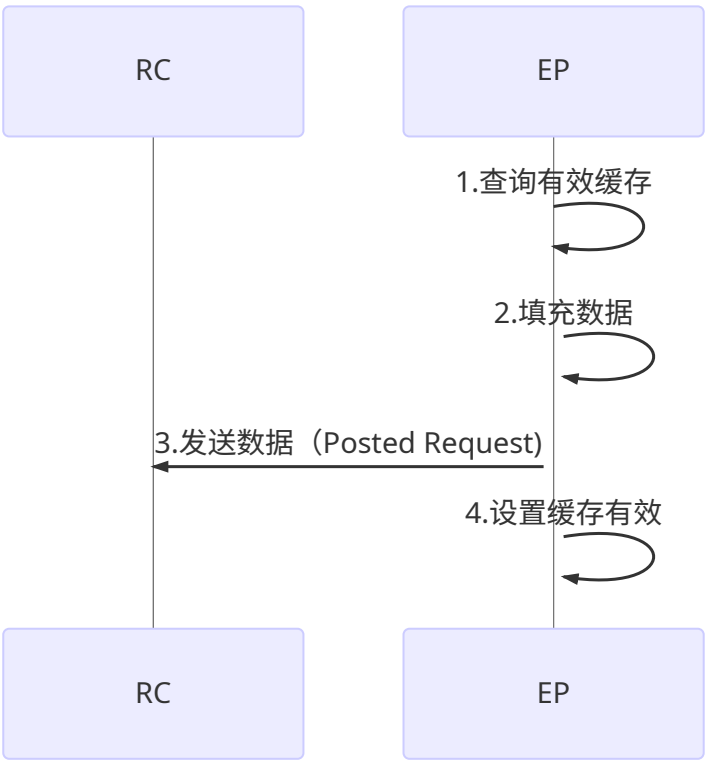
`rk_pcie_get_buff`暂不支持阻塞模式，所以需要通过调用sleep来循环等待。

```
struct pcie_buff_node *node = rk_pcie_get_buff(rk_handle, E_SEND);
if(node) {
    node->size = rk_pcie_get_buff_max_size(rk_handle, E_SEND);
    //操作buff 写入数据
    rk_pcie_send_buff(ctx->rk_handle, node);
} else {
    usleep(1000);
}
```

RC 向 EP 发送数据原理：



EP 向 RC 发送数据原理：



5.3.3 接收数据

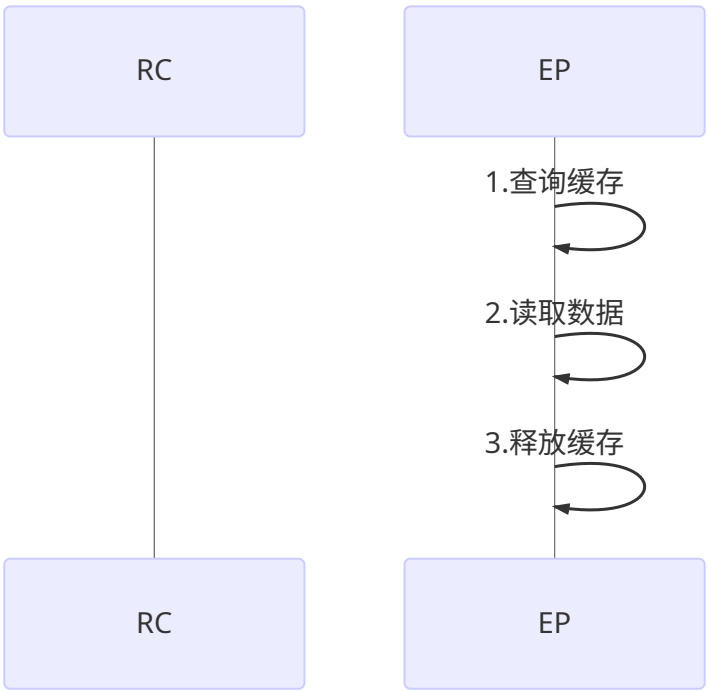
接收数据可以分为3个步骤

1. 查询是否有可用BUFF。
2. 获取可用BUFF，并读取数据。
3. 释放BUFF。

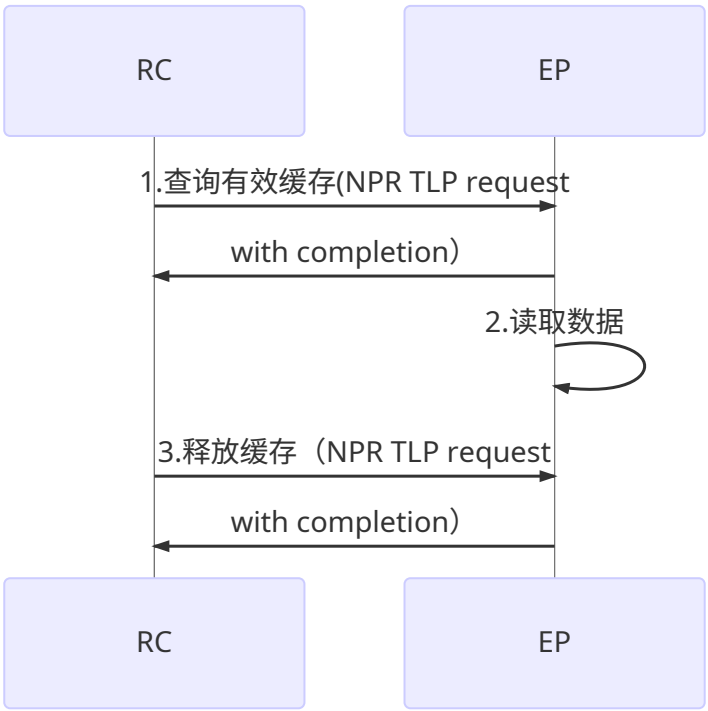
   接收BUFF和发送BUFF内部实现类似，接收BUFF是根据初始化`recv_buff_num`来配置系统中BUFF个数。具体行为可以参考发送数据章节。BUFF可用数据的大小就是`node->size`。

```
struct pcie_buff_node *node = rk_pcie_get_buff(rk_handle, E_RECV);
if(node) {
    //操作buff 读取数据
    rk_pcie_release_buff(rk_handle, node);
} else {
    usleep(1000);
}
```

EP 接收数据原理：



RC 接收数据原理：



5.3.4 BUFF配置

1. RC端发送到EP的BUFF配置

解码卡场景下，RC端是发送裸码流到EP，单路的数据量较少。可以申请`4KB*video chn`的BUFF大小。每路视频流单次传输4KB的大小，这样可以保证每次传输的视频流都可以更新数据，并且每次数据量不会过小导致浪费PCIE带宽。

BUFF数据建议配置3-5个，方便轮转使用。最少必须配置三个，如果有对BUFF做其它处理需要增加BUFF数量，不然会导致轮转卡住影响传输速率。

多线程传输可以适当加大BUFF数量。

2. EP发送到RC的BUFF配置。

   EP发送到RC的数据，正常是解码后的数据一般比较大，比如1080p解成YUV422的单帧数据大概是3M的大小。单个BUFF可以配置为3MB-6MB大小，每次传输1路或者2路解码后的数据。因为需要多个BUFF轮转，单个BUFF不宜配置太大会占用太大的内存空间。

5.4 用户层内存配置

RC/EP均支持多种内存申请方式：

- RC端：

1. pcie-rkep驱动申请内存，SDK通过mmap映射获取。

   使能配置：CONFIG_PCIE_FUNC_RKEP_USERPAGES=y（默认不使能）

   修改内存大小：修改驱动pcie-rkep.c RKEP_USER_MEM_SIZE宏（默认是64M）

2. 通过HugePages 申请内存。(推荐方式)

3. 通过DRM驱动申请内存，需要在dts中对CMA预先分配。

   DMA内存大小修改设备树：arch/arm64/boot/dts/rockchip/rk3588-linux.dtsi

```
@@ -71,7 +71,7 @@ reserved-memory {
cma {
    compatible = "shared-dma-pool";
    reusable;
    reg = <0x0 0x50000000 0x0 0x10000000>; //从地址0x50000000开始256MB大小
    linux,cma-default;
};
```

4. dummy方式，数据buf交由用户申请管理，框架只负责对轮转node管理，不申请数据buf。当前仅支持Windows系统。

- EP端：

1. 通过DRM驱动申请内存，需要在dts中对CMA预先分配。

   DMA内存大小修改设备树：arch/arm64/boot/dts/rockchip/rk3588-linux.dtsi

```
@@ -71,7 +71,7 @@ reserved-memory {
cma {
    compatible = "shared-dma-pool";
    reusable;
    reg = <0x0 0x50000000 0x0 0x10000000>; //从地址0x50000000开始256MB大小
    linux,cma-default;
};
```

2. 通过HugePages 申请内存。(推荐方式)

不同内存申请方式可以在初始化Task通过属性来指定

- RC端：E_MEM_TYPE_HUGEPAGE / E_MEM_TYPE_RKDRM / E_MEM_TYPE_RKEP
- EP端：E_MEM_TYPE_HUGEPAGE / E_MEM_TYPE_RKDRM

说明：

使用 HugePages 方式申请需要对系统进行配置，可以查看 HugePage 章节 进行配置。

5.5 DMA PRPS 链表传递支持

chips_connect V3.3.0 新增了 DMA PRPS（Physical Region Page Structures）链表传递支持，用于优化非连续内存传输性能。该功能允许 RC 端将非连续物理内存的 PRPS 链表直接传递给 EP 端，EP 端解析 PRPS 链表后获取实际的物理地址列表进行 DMA 传输。

PRPS 页结构

PRPS 链表以页（4KB）为单位组织，每个 PRPS 页包含：

```
#define PRPS_PAGE_SIZE              4096
#define PRPS_NEXT_PHY_PTR_OFFSET    8    /* 距离页尾的偏移（物理地址）*/
#define PRPS_NEXT_VIR_PTR_OFFSET    16   /* 距离页尾的偏移（虚拟地址）*/
#define PRPS_MAX_ITEMS_PER_PAGE     254  /* 每页最多 254 个 PRPS 项 */
```

PRPS 项结构

```
struct pcie_prps_item {
    cc_u64 phy_addr;    /* 物理地址 */
    cc_u32 size;        /* 数据块大小 */
    cc_u32 reserved;    /* 保留字段 */
};
```

PRPS 链表组织

PRPS 链表通过页尾的指针链接多个 PRPS 页：

```
+------------------+     +------------------+     +------------------+
|  PRPS Page 0     |     |  PRPS Page 1     |     |  PRPS Page N     |
| +--------------+ |     | +--------------+ |     | +--------------+ |
| | PRPS Item 0  | |     | | PRPS Item 0  | |     | | PRPS Item 0  | |
| | PRPS Item 1  | |     | | PRPS Item 1  | |     | | PRPS Item 1  | |
| | ...          | |     | | ...          | |     | | ...          | |
| | PRPS Item 253| |     | | PRPS Item 253| |     | | PRPS Item 253| |
| | Next Vir Ptr | |     | | Next Vir Ptr | |     | | Next Vir Ptr | |
| | Next Phy Ptr | |---->| | Next Phy Ptr | |---->| | Next Phy Ptr | |---> 0
| +--------------+ |     | +--------------+ |     | +--------------+ |
+------------------+     +------------------+     +------------------+
```

说明：

- 每个 PRPS 页最多包含 254 个数据项（占用 254 * 16 = 4064 字节）
- 页尾 8 字节存储下一个 PRPS 页的物理地址指针
- 页尾 16 字节存储下一个 PRPS 页的虚拟地址指针
- 最后一个 PRPS 页的 Next Ptr 为 0

适用场景

PRPS 链表传递特别适用于以下场景：

1. **大块数据传输**：单次传输数据量 > 1MB
2. **非连续内存环境**：系统内存碎片化严重，难以申请大块连续内存
3. **多任务并发**：多个 Task 同时运行，每个 Task 需要独立的大内存缓冲区
4. **高内存利用率要求**：需要最大化利用系统内存，减少碎片

RC 端配置

PRPS 链表传递功能在 RC 端自动启用（当满足条件时）：

- 自动启用条件：当使用非连续内存申请内存时，会自动配置 PRPS链表模式
- 配置参数：无需额外配置，框架自动管理 PRPS 链表的创建和传递

限制与注意事项

1. **版本依赖**：需要 RC 和 EP 端的 chips_connect 版本都 >= V3.3.0
2. **链表传输模式**：需要启用 `use_dma_ll` 配置
3. **内存开销**：PRPS 链表本身需要额外的内存（每 4KB 数据约需 16 字节 PRPS 项）
4. **最大支持**：单缓冲区最大支持 8K 个 PRPS 项（约 32MB 数据）

```
#define PCIE_DMA_LL_MAX_NUM_PER_BUFFER 0x2000 /* 8K * 4K = 32MB */
```

5.6 用户层驱动优点

把PCIe的数据传输服务驱动提到用户层主要是方便使用和性能优化两方面考虑。

使用方面：

- 基于PCIe设备驱动，方便不同平台的移植；
- 方便直接跟应用对接，可以直接调用RK提供的rockit接口，或者第三方gstreamer接口等；

对比kernel接口驱动，应用层驱动在性能方面优化主要有：

- 使用轮询替换中断，减少频繁中断导致的CPU模式切换开销；
- 减少kernel和userspace切换导致的CPU模式切换开销；
- 建议绑定CPU，减少多核调度开销；
- 去掉kernel space和userspace之间内存拷贝开销，可以做到memory zero-copy；
- 使用系统Huge Page memory，减少TLB cache miss;

5.7 用户层多个独立应用扩展

RK chips_connect用户层应用基础为Task：

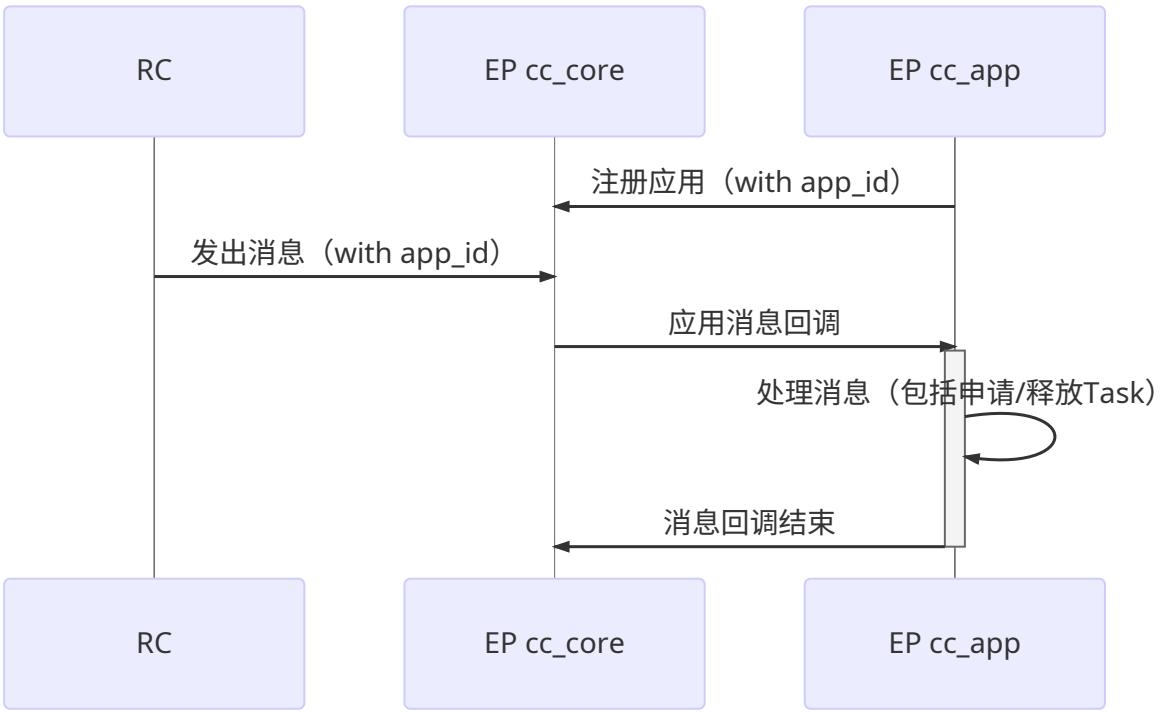
- Task之间相互独立
  - RC用户消息相互独立，RC Task支持多进程开发
  - EP用户消息统一管理，EP Task不支持多进程
- Task内支持多线程
- Task ID先到先得、无法指定具体ID

由于默认参考案例应用与Task等资源未绑定，所以无法实现多个独立应用并行，建议基于带应用程序ID的用户消息划分消息、Task归属，扩展多个独立应用。例如RK RT-Thread chips_connect多应用解决方案：

1. 定义用户消息规则，添加app管理：

```
struct task_user_cmd_st {
    unsigned int app_id;
    unsigned int cmd;
    unsigned char data[4];
};
```

2. EP应用程序注册应用，unique app_id
3. RC发出消息，unique app_id
4. EP主程序根据app_id触发应用消息回调
5. EP应用程序处理消息，包括申请/释放Task



6. Chips Connect配置

6.1 EP卡用户层设备驱动

EP卡用户层设备驱动指RC端运行的EP function驱动，包括内核模块和用户层APP。

6.1.1 Using-RC-DMA模式

Using-RC-DMA模式只能工作在EP卡对接RK RC设备情况下，该模式下可以让PCIE速率达到最大化。

默认配置下只会使用EP端的DMA，Using-RC-DMA模式的原理就是把RC端的DMA使用起来。EP写数据到RC，则调用EP的DMA，RC写数据到EP，则调用RC的DMA，互不干扰。

RC端DMA有限制只能搬运到BAR有做 memory inbound 映射的物理空间，所以这里把BAR2给RC DMA使用。默认大小是64M，可以在EP端dts中修改配置大小。这个内存只是RC写数据到EP的时候使用，也就是初始化阶段配置的`send_buff_size`大小，最小不能小于`send_buff_size`。

使能模式

RC以及EP设备驱动不需要修改，只需要RC上层应用初始化阶段参数配置

```
dev_attr.using_rc_dma = 1; //使用RC DMA
dev_attr.rc_dma_chn = 1; //通道1，支持0/1
```

使能成功后可以在RC应用看到如下LOG:

```
rk pcie version: 30301
using rc dma: 1, chn: 1   //使能RC DMA，通道1
open rkep: /dev/pcie-rkep-0000:01:00.0 Success.
```

EP应用创建Task会有如下LOG：

```
task create id: 1
using rc dma: 1
```

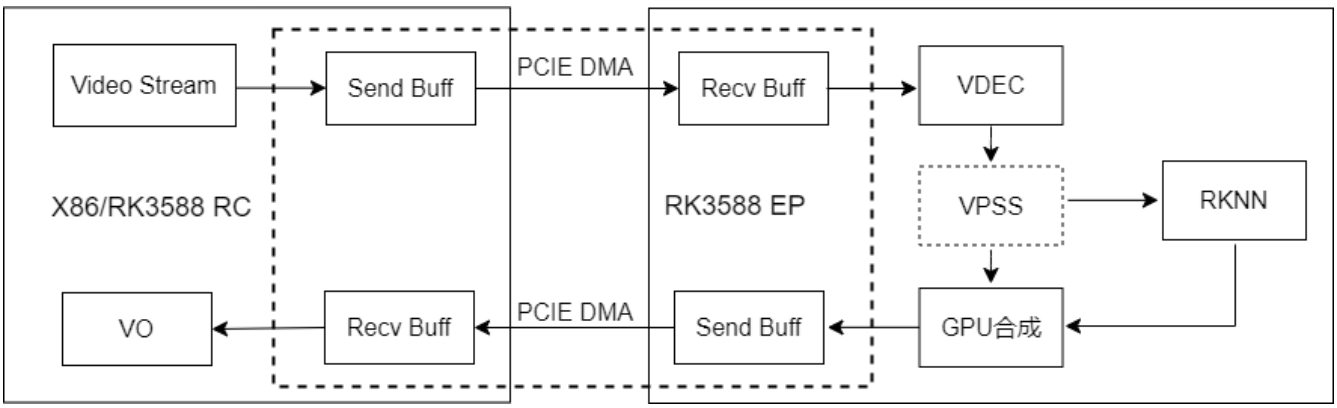
推荐配置

- 单EP场景：推荐使能using_rc_dma，并且配置为通道0。
- 多EP场景：推荐关闭using_rc_dma，使用EP各自的DMA搬运数据，可以让每个EP的传输速率保持大致相同。

7. Chips Connect典型业务场景

7.1 EP视频加速卡

视频加速卡是将RC的视频流通过PCIE总线传输给EP解码并处理后再通过PCIE总线传给主片显示。基本的数据流处理如下图所示：



码流发送端首先从视频流中获取码流数据，将其拷贝至准备好的 send stream buffer 中，然后通过 PCIe 的 DMA 将码流数据发送到 PCIe 对端的 recv stream buffer 中，对端再将码流取出通过VDEC解码送入VPSS，VPSS输出2路数据，一路送入GPU合成，一路数据送入RKNN进行处理。把合成后的数据拷贝至准备好的send stream buffer 中，然后通过 PCIe 的 DMA 将数据发送到 PCIe 对端的recv stream buffer 中，对端将数据取出送入VO模块显示。

SDK已经实现发送端和接收端的 stream buffer，初始化阶段配置参数即可使用，用户只需要关注业务流程。

8. Chips Connect验收标准

首次使用PCIE EP开发包建议按照如下流程确认软件、硬件、基础业务流程正常后，再开始自定义功能开发。

8.1 开发包清单

目录结构介绍

```
.
├── bootloader                    # pcie ep bootloader源码工程
├── build                         # build编译脚本
├── core
│   ├── api                       # sdk对外api，包含rc和ep
│   ├── component                 # pcie组件，主要是负责buf管理，消息通知等
│   └── framework                 # 框架层，主要负责和OS交互，对上提供系统能力以及PCIE功能
├── docs                          # PCIe EP SDK文档介绍
├── examples
│   ├── pcie_camera_test          # EP卡典型业务，抓取EP Camera数据数据处理，仅android端使用
│   ├── pcie_speed_test           # EP卡典型业务，PCIE传输速率测试Demo
│   ├── pcie_video_test           # EP卡典型业务，视频卡Demo
│   ├── pcie_download_test        # EP卡RC下载固件启动方案Demo
│   ├── pcie_auto_test            # EP卡功能自动化测试Demo
│   ├── pcie_loopback_test        # EP卡RC和EP传输速率环回测试Demo
│   ├── pcie_userbuf_test         # EP卡使用外部用户buf进行测试Demo
│   ├── pcie_vuart_rc             # EP卡PCIE虚拟串口，RC端模拟EP端控制台
│   ├── pcie_upgrade_tool         # EP卡升级工具
│   └── windows                   # EP卡Windows测试Demo
├── win_driver                    # Windows内核驱动源码，基于KMDF框架
├── images                        # 功能验证固件
├── patch*                        # 补丁目录
└── third-party                   # 第三方依赖
```

开发包主要目录介绍：

core/framework目录对基础功能实现，包括内存分配管理、日志功能、linked_list、DMA、PCIe CONFIG等RC和EP通用组件。

- framework/core/pcie：实现DMA传输，负责和芯片DMA配置交互。
- framework/core/mem：实现不同配置内存申请释放。
- framework/osal：对os接口实现封装，对上层屏蔽不同os差异。
- framework/base：实现基础数据结构，比如链表、队列等。

core/component目录封装RC和EP接口实现，对内实现buf管理，消息通知，数据传输等，对外开放特定API方便不通过OS调用开发。该目录由三部分组成：

- ep_pcie：负责PCIE EP部分buf管理，消息通知，数据传输相关实现。
- rc_pcie：负责PCIE RC部分buf管理，消息通知，数据传输相关实现。
- cc_pcie_type.h 和 cc_cmp_pcie_type.h : 对外提供统一的结构体和数据类型，方便rc和ep通信。

core/api目录下封装了不同系统用户接口层的实现，对外开放特定API方便用户开发，该目录下由两部分组成：

- ep：RK EP只需要支持arm环境，所以该部分只是对component/ep_pcie接口封装，没有实现额外的逻辑。
- rc：只是对component/rc_pcie接口封装，没有实现额外的逻辑。

build：目录实现对sdk编译构建，支持linux、android、windows。其中build/windows包含Visual Studio解决方案(rk_projects.sln)，包含三个项目：rkep_driver(内核驱动)、rk_cc(静态库)、examples(测试应用)。编译输出到install目录。

docs：标准EP板卡说明文档。

examples：提供不同测试Sample，Sample介绍以及操作说明查看对应目录下的README。

images：目录下存放可直接用于基础功能测试的固件以及Sample。

patch：*SDK包对应的补丁，支持linux和android系统。补丁说明请查看`patch*/README`。

win_driver：Windows内核驱动源码目录，基于KMDF框架开发。包含驱动入口(RkAcceleratorCard.c)、设备初始化(Init.c)、中断处理(IsrDpc.c)、IOCTL控制(Control.c)、PCIe操作(rk_pcie.c)等核心模块，以及操作系统抽象层(osal)。支持RK35xx(VEN_1D87&DEV_356A)和RK182x(VEN_1D87&DEV_182A)设备。

8.2 补丁更新｜一键生效“更新包”源码

测试目标

标准卡驱动版本升级，支持一键添加“更新包”目录下的补丁。

测试方法

RK3588：

```
./ep_patch.sh release patch-kernel5.10/更新包 ./path-to-dst/
```

RK3568：

```
./ep_patch.sh release patch-kernel4.19/更新包 ./path-to-dst/
```

其他说明：

- 生成补丁包：

```
./ep_patch.sh release ./.. patch-kernel5.10/更新包
cd ./../kernel/
git checkout develop-4.19
cd -
./ep_patch.sh kernel ./.. patch-kernel4.19/更新包
```

8.3 功能｜关键日志信息

8.3.1 RC端lspci关键信息

确认以下信息：

- bar使能且正常分配
- MSI使能且正常分配
- LnkCap，Gen3x4支持、ASPM not supported
- LnkSta，Gen3x4状态
- Kernel driver in use: pcie-rkep

log示例：

```
01:00.0 Class 1200: Device 1d87:356a (rev 01)
        Control: I/O- Mem+ BusMaster+ SpecCycle- MemWINV- VGASnoop- ParErr- Stepping- SERR-
FastB2B- DisINTx+
        Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort-
>SERR- <PERR- INTx-
        Latency: 0
        Interrupt: pin A routed to IRQ 157
        Region 0: Memory at f0400000 (32-bit, non-prefetchable) [size=4M]
        Region 2: Memory at 900000000 (64-bit, prefetchable) [size=64M]
        Region 4: Memory at f0200000 (32-bit, non-prefetchable) [size=1M]
        Capabilities: [40] Power Management version 3
                Flags: PMEClk- DSI- D1+ D2+ AuxCurrent=375mA PME(D0+,D1+,D2-,D3hot+,D3cold-)
                Status: D0 NoSoftRst+ PME-Enable- DSel=0 DScale=0 PME-
        Capabilities: [50] MSI: Enable+ Count=4/32 Maskable+ 64bit+
                Address: 00000000fe670040  Data: 0000
                Masking: fffffff0  Pending: 00000000
        Capabilities: [70] Express (v2) Endpoint, MSI 08
                DevCap: MaxPayload 256 bytes, PhantFunc 0, Latency L0s unlimited, L1
unlimited
                        ExtTag+ AttnBtn- AttnInd- PwrInd- RBE+ FLReset- SlotPowerLimit 0.000W
                DevCtl: Report errors: Correctable- Non-Fatal- Fatal- Unsupported-
                        RlxdOrd+ ExtTag+ PhantFunc- AuxPwr- NoSnoop-
                        MaxPayload 256 bytes, MaxReadReq 256 bytes
                DevSta: CorrErr- UncorrErr- FatalErr- UnsuppReq- AuxPwr- TransPend-
                LnkCap: Port #0, Speed 8GT/s, Width x4, ASPM not supported, Exit Latency L0s
<4us, L1 <16us
                        ClockPM- Surprise- LLActRep- BwNot- ASPMOptComp+
                LnkCtl: ASPM Disabled; RCB 64 bytes Disabled- CommClk-
                        ExtSynch- ClockPM- AutWidDis- BWInt- AutBWInt-
                LnkSta: Speed 8GT/s, Width x4, TrErr- Train- SlotClk+ DLActive- BWMgmt-
ABWMgmt-
                DevCap2: Completion Timeout: Not Supported, TimeoutDis+, LTR+, OBFF Via
message/WAKE#
                DevCtl2: Completion Timeout: 50us to 50ms, TimeoutDis-, LTR+, OBFF Disabled
                LnkCtl2: Target Link Speed: 8GT/s, EnterCompliance- SpeedDis-
                         Transmit Margin: Normal Operating Range, EnterModifiedCompliance-
ComplianceSOS-
                         Compliance De-emphasis: -6dB
                LnkSta2: Current De-emphasis Level: -6dB, EqualizationComplete+,
EqualizationPhase1+
                         EqualizationPhase2+, EqualizationPhase3+, LinkEqualizationRequest-
        Capabilities: [b0] MSI-X: Enable- Count=128 Masked-
                Vector table: BAR=4 offset=00020000
                PBA: BAR=4 offset=00028000
        Capabilities: [100 v2] Advanced Error Reporting
                UESta:  DLP- SDES- TLP- FCP- CmpltTO- CmpltAbrt- UnxCmplt- RxOF- MalfTLP-
ECRC- UnsupReq- ACSViol-
                UEMsk:  DLP- SDES- TLP- FCP- CmpltTO- CmpltAbrt- UnxCmplt- RxOF- MalfTLP-
ECRC- UnsupReq- ACSViol-
                UESvrt: DLP+ SDES+ TLP- FCP+ CmpltTO- CmpltAbrt- UnxCmplt- RxOF+ MalfTLP+
ECRC- UnsupReq- ACSViol-
```

```
                CESta:  RxErr- BadTLP- BadDLLP- Rollover- Timeout- NonFatalErr-
                CEMsk:  RxErr- BadTLP- BadDLLP- Rollover- Timeout- NonFatalErr+
                AERCap: First Error Pointer: 00, GenCap+ CGenEn- ChkCap+ ChkEn-
        Capabilities: [148 v1] #19
        Capabilities: [168 v1] Address Translation Service (ATS)
                ATSCap: Invalidate Queue Depth: 00
                ATSCtl: Enable-, Smallest Translation Unit: 00
        Capabilities: [178 v1] Page Request Interface (PRI)
                PRICtl: Enable- Reset-
                PRISta: RF- UPRGI- Stopped+
                Page Request Capacity: 00000001, Page Request Allocation: 00000000
        Capabilities: [188 v1] Latency Tolerance Reporting
                Max snoop latency: 0ns
                Max no snoop latency: 0ns
        Capabilities: [190 v1] L1 PM Substates
                L1SubCap: PCI-PM_L1.2+ PCI-PM_L1.1+ ASPM_L1.2+ ASPM_L1.1+ L1_PM_Substates+
                          PortCommonModeRestoreTime=10us PortTPowerOnTime=10us
        Capabilities: [1a0 v1] #16
        Capabilities: [1d0 v1] Vendor Specific Information: ID=0002 Rev=4 Len=100 <?>
        Capabilities: [2d0 v1] Vendor Specific Information: ID=0006 Rev=0 Len=018 <?>
        Capabilities: [2e8 v1] #15
        Kernel driver in use: pcie-rkep
```

8.3.2 EP端spl.bin启动 log

确认以下信息：

- kernel阶段打印“already linkup”

log示例：

```
... #spl关键打印

U-Boot SPL board init
RKEP: Init PCIe EP
RKEP: 188259 - PHY Mode 0x4
RKEP: 189537 - RKEP: GRF:904=7, a04=7...
RKEP: 190138 - RKEP: GRF:904=7, a04=f...
RKEP: 190487 - RKEP: GRF:904=f, a04=f...
RKEP: 190827 - RefClock in common clock_mode
RKEP: 200593 - BAR0: 0x3c000000
RKEP: 200864 - BAR2: 0x010000000
RKEP: 201247 - init PCIe fast Link up
RKEP: 218243 - Link up 230011
RKEP: 259699 - Done

... #内核关键打印

[00-06-34.831][    3.148198] rk-pcie-ep fe150000.pcie: bar0: assigned [0x3c000000-3c3fffff]
[00-06-34.834][    3.149015] rk-pcie-ep fe150000.pcie: bar2: assigned [0x40000000-43ffffff]
[00-06-34.834][    3.149027] rk-pcie-ep fe150000.pcie: no vpcie3v3 regulator found
[00-06-34.837][    3.149046] rk-pcie-ep fe150000.pcie: already linkup
```

```
[00-06-34.837][    3.149320] rk-pcie-ep fe150000.pcie: register misc device pcie_ep
```

8.3.3 EP端pcie.bin启动 log

确认以下信息：

- 记录pcie.bin运行时间

版本	耗时
V1.00	360ms

- spl阶段打印“already link up”
- kernel阶段打印“already linkup”

log示例：

```
#pcie.bin关键打印
PCIe_EP Release Time: Jan  2 2025 V2.20
Gen3x4
phyMode4
HW
controller initialized
GRF:904=0007, a04=0007...
GRF:904=004F, a04=004F...
phy locked
phy initialized
vid=0x1D87 did=0x356A
ms=000C
Linking, ltssm:
00000001
00000002
00000007
00210022
00210023
00230011
Link up
rst_fun1
Link stable, ltssm
00230011
out

... #spl关键打印

U-Boot SPL board init
RKEP: Init PCIe EP
RKEP: already link up

... #内核关键打印
```

```
[00-06-34.831][    3.148198] rk-pcie-ep fe150000.pcie: bar0: assigned [0x3c000000-3c3fffff]
[00-06-34.834][    3.149015] rk-pcie-ep fe150000.pcie: bar2: assigned [0x40000000-43ffffff]
[00-06-34.834][    3.149027] rk-pcie-ep fe150000.pcie: no vpcie3v3 regulator found
[00-06-34.837][    3.149046] rk-pcie-ep fe150000.pcie: already linkup
[00-06-34.837][    3.149320] rk-pcie-ep fe150000.pcie: register misc device pcie_ep
```

8.4 功能｜固件烧录

请阅读文档《Rockchip_Developer_Guide_PCIe_EP.md》中"EP卡固件烧录"及"单EP卡对接通用RC"章节，确认是否为需要快速启动的产品形态，并确认所使用存储方案并做对应PC升级烧录验证，主要包括：

- 单存储
- 双存储
- EP Boot

详细参考"EP卡固件编译"章节。

8.5 性能｜用户态测试Demo - pcie_speed_test

8.5.1 测试目标

验证业务基础，主要包括：

- RC端wired DMA to EP传输、DMA传输速率及双向DMA传输
- RC端local DMA to RC传输、DMA传输速率及双向DMA传输
- bar映射、访问正常
- ELBI 工作正常，支持RC 触发中断事件通知EP
- MSI 工作正常，支持EP 触发中断事件通知RC

根据需求验证：

- 如果选用支持DMA的RK RC，支持RC端Local DMA to EP传输，DMA传输速率
- firmware download

8.5.2 测试方法

详细参考examples/pcie_speed_test/README.md文档。

8.5.3 RK3588 Gen3x4测试结果

测试设备

RC：RK3588-EVB1-LP4X-V10 PCIe 3.0 4 lanes大核2.4G小核1.8G，CPU/Dram 定高频

EP：RK3588-EVB4-LP4X-V10 PCIe 3.0 4 lanes大核2.4G小核1.8G，CPU/Dram 定高频

标准 EP 版本

Rockchip_PCIE_EP_Standard_Card_20231215.tar.gz

测试参数：

- 使用 EP DMA 引擎
- task_num = 1

测试结果

DMA大小	64B	256B	4K	64KB	1MB	4MB
RC 读（DMA 写）	0.16MB/s	0.65MB/s	10.4MB/s	165.8MB/s	1898.7MB/s	3271.3MB/s
RC 写（DMA 读）	0.17MB/s	0.73MB/s	11.0MB/s	171.1MB/s	1630.6MB/s	1735.6MB/s

说明：

- 如果是RC发起测试，且使用默认的EP DMA引擎，写测试实际为EP DMA读，读测试实际为EP DMA写
- 测试粒度越小，调度参与越多，性能越差，但可以通过开多线程提高整体性能

8.5.4 RK3588 Gen2x1测试结果

测试设备

RC：RK3588-EVB1-LP4X-V10 PCIe 3.0 4 lanes大核2.4G小核1.8G

EP：RK3588-EVB4-LP4X-V10 PCIe 3.0 4 lanes（降为Gen2 1 lanes）大核2.4G小核1.8G

标准EP版本

Rockchip_PCIE_EP_Standard_Card_20231215.tar.gz

测试参数：

- 使用EP DMA引擎
- task_num = 1

测试结果

DMA大小	64B	256B	4K	64KB	1MB	4MB
写	0.16MB/s	0.64MB/s	10.2MB/s	147.6MB/s	394.0MB/s	396.2MB/s
读	0.17MB/s	0.70MB/s	11.0MB/s	149.2MB/s	384.0MB/s	384.1MB/s

说明：

- 如果是RC发起测试，且使用默认的EP DMA引擎，写测试实际为EP DMA读，读测试实际为EP DMA写
- 测试粒度越小，调度参与越多，性能越差，但可以通过开多线程提高整体性能

8.5.5 RK1820 Gen2x1测试结果

测试设备

RC：RK3588-EVB1-LP4X-V10 PCIe 3.0 4 lanes大核2.4G小核1.8G

EP：RK1820 PCIe 2.0 1 lanes

标准EP版本

RC - Rockchip_PCIE_EP_Standard_Card_20250523_V3.1.0.tar.gz

EP - RK RT-Thread SDK develop-v5.1.x 分支

测试参数：

- 使用EP DMA引擎
- task_num = 1

RK1820 DRAM 内存测试结果

DMA大小	64B	256B	4K	8K	64KB	512KB
RC 写（EP DMA 读）	3MB/s	10MB/s	136MB/s	188MB/s	303MB/s	317MB/s
RC 读（EP DMA 写）	3MB/s	11MB/s	143MB/s	237MB/s	363MB/s	382MB/s

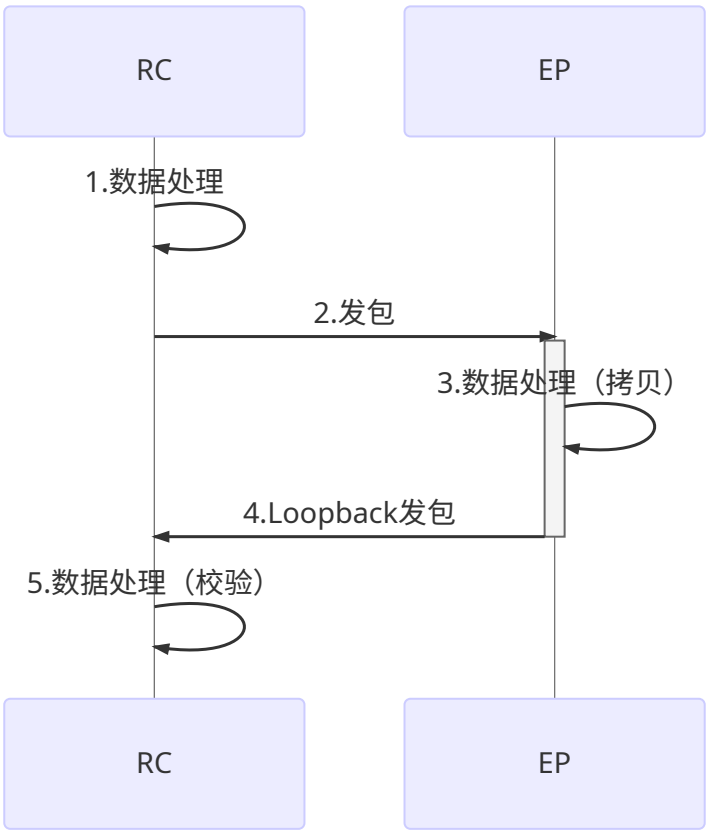
说明：

- 如果是RC发起测试，且使用默认的EP DMA引擎，写测试实际为EP DMA读，读测试实际为EP DMA写
- 测试粒度越小，调度参与越多，性能越差，但可以通过开多线程提高整体性能
- 相较于EP Linux OS，RT-Thread OS为实时系统，用户层程序可快速响应中断，所以小粒度数据传输依旧保持较高吞吐

8.6 性能｜用户态测试Demo - pcie_loopback_test

8.6.1 测试目标

统计典型发包回包场景，统计串行流程的响应耗时时长。



8.6.2 测试方法

详细参考examples/pcie_speed_test/README.md文档。

8.6.3 RK3588 Gen2x1测试结果

测试设备

RC：RK3588-EVB1-LP4X-V10 PCIe 3.0 4 lanes大核2.4G小核1.8G

EP：RK3588-EVB4-LP4X-V10 PCIe 3.0 4 lanes（降为Gen2 1 lanes）大核2.4G小核1.8G

标准EP版本

Rockchip_PCIE_EP_Standard_Card_20231215.tar.gz

测试参数：

- 使用EP DMA引擎
- task_num = 1

测试结果

DMA大小	4K
min/max	105us/724us

说明：

- 如果是RC发起测试，且使用默认的EP DMA引擎，写测试实际为EP DMA读，读测试实际为EP DMA写
- 测试结果为测试1秒后统计值，排除初始化过程的偏差影响
- 测试结果为持续发包下的best case结果，请根据需求调整间隙和其他负载来构建worst case

8.6.4 RK1820 Gen2x1测试结果

测试设备

RC：RK3588-EVB1-LP4X-V10 PCIe 3.0 4 lanes大核2.4G小核1.8G

EP：RK1820 PCIe 2.0 1 lanes

标准EP版本

RC - Rockchip_PCIE_EP_Standard_Card_20250523_V3.1.0.tar.gz

EP - RK RT-Thread SDK develop-v5.1.x 分支

测试参数：

- 使用EP DMA引擎
- task_num = 1

RK1820 DRAM 内存测试结果

DMA大小	4K
min/max	83us/139us

说明：

- 如果是RC发起测试，且使用默认的EP DMA引擎，写测试实际为EP DMA读，读测试实际为EP DMA写
- 测试结果为测试1秒后统计值，排除初始化过程的偏差影响
- 测试结果为持续发包下的best case结果，请根据需求调整间隙和其他负载来构建worst case

8.7 兼容性 | 枚举

测试目标

验证EP板卡在不同RC上探测、识别、枚举的兼容性和健壮性。

测试方法

RK EP主要对接的RC：

EP设备	RC设备	冷启	休眠唤醒	热启	确认方法
RK3588 EVB4 V11	RK3588 EVB1 V10	20/20	20/20	20/20	Linux：lspci | grep 356a
RK3588 EVB4 V11	PC ubuntu	20/20	20/20	20/20	Linux：lspci | grep 356a
RK3588 EVB4 V11	PC windows	20/20	20/20	20/20	设备管理器确认是否有对应设备
RK3568 iotest设备	RK3588 EVB1 V10	20/20	20/20	20/20	Linux：lspci | grep 356a
RK3568 iotest设备	PC ubuntu	20/20	20/20	20/20	Linux：lspci | grep 356a

说明：

- 实际产品尽可能多的覆盖目标RC设备
- 冷启：PC断电、上电、按下POWER按键
- 休眠唤醒：
  - RK RC：POWER按键进出休眠唤醒
  - ubuntu：桌面版本通过可以通过界面按键进入，其他场景自行确认
  - Windows：CTRL+L组合按键进入，其他场景自行确认
- 热启：
  - RK3588：进入root权限，shell界面输入reboot指令
  - ubuntu：进入root权限，shell界面输入reboot指令
  - Windows：桌面下，ALT+F4组合键弹出复位选项，选择复位，部分PC支持直接按POWER按键复位

8.8 兼容性 | EP复位RC检测并重新枚举

测试目标

测试”EP单独复位“场景下的业务健壮性，主要测试以下内容：

- PCIe remove/rescan 功能
- 通讯是否正常恢复
- 业务是否正常运行

测试方法

以下测试需要上述几项功能测试都正常情况下进行，并且编译examples/pcie_auto_test可执行文件，参考examples/pcie_auto_test/README.md文档部署测试环境。

- PCIe remove/rescan 功能
  - RC端执行`./pcie_auto_test_rc 2 0`以及`./pcie_auto_test_rc 3 0`选择第0个设备进行Remove以及Rescan操作。

- 通讯是否正常恢复
- 业务是否正常运行
    - RC端执行`./pcie_auto_test_rc 0 0`选择第0个设备进行API稳定性操作。

8.9 功能 | OTA

请阅读文档《Rockchip_Developer_Guide_PCIe_EP.md》中"EP卡OTA"及"单EP卡对接通用RC"章节，确认是否为需要快速启动的产品形态，并确认所使用存储方案并做对应OTA验证，主要包括：

- 单存储
- 双存储
- EP Boot

详细参考"EP卡OTA"章节。

9. Chips Connect异常处理

9.1 RC端x86平台执行初始化PCIE报如下错误mmap failed, Operation not permitted

```
root@rk:/home/rc-sample# ./pcie_test 10240
resource path = /sys/bus/pci/devices/0000:01:00.0/enable
resource path = /sys/bus/pci/devices/0000:01:00.0/resource0
mmap failed, ret = 0, Operation not permitted
ep dev num : 1
Segmentation fault (core dumped)
```

答：这个是因为ubuntu从20.04开始增加linux 内核锁定功能，如果在bios使能安全启动功能，会打开内核锁定导致mmap失败。需要进入bios关闭安全启动即可正常。

9.2 HugePage配置

RK平台Hugepage大小支持2M、32M、1GB，其它平台支持大小以实际系统为主。单个Hugepage大小配置需要大于SDK中单个发送buff以及单个接收buff的大小，Hugepage数量需要超过所需要的Buff数量。

比如：5个发送Buff，单个Buff是4M，5个接收Buff，单个Buff是8M。

这种情况下就不能配置Hugepage为2M，最小配置需要32M

发送Buff需要的Hugepage个数 = (int) (5 / (32M / 4M)) + 1 = 1

接收Buff需要的Hugepage个数 = (int) (5 / (32M / 8M)) + 1 = 2

则最终配置为`default_hugepagesz=32M, hugepagesz=32M, hugepages=3`

9.2.1 X86平台

查看Hugepagesize大小

```
rk@debian:~$ cat /proc/meminfo | grep Huge
HugePages_Total:       2
HugePages_Free:        2
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:    2048 kB
Hugetlb:         4096 kB
```

修改大小为1GB

- 修改cmdline参数，编写/etc/default/grub, 添加default_hugepagesz=1G

  ```
  GRUB_CMDLINE_LINUX_DEFAULT="quiet default_hugepagesz=1G hugepagesz=1G hugepages=1"
  GRUB_CMDLINE_LINUX=""
  ```

- 生成配置

  ```
  /usr/sbin/grub-mkconfig -o /boot/grub/grub.cfg
  ```

- 重启设备 reboot 或者 systemctl poweroff
- 查看size和count，`cat /proc/meminfo | grep Hugepagesize`

  ```
  rk@debian:~$ cat /proc/meminfo | grep Huge
  HugePages_Total:       1
  HugePages_Free:        1
  HugePages_Rsvd:        0
  HugePages_Surp:        0
  Hugepagesize:    1048576 kB
  Hugetlb:         1048576 kB
  ```

9.2.2 RK平台

内核开启Hugepage配置

```
+++ b/arch/arm64/configs/rockchip_linux_defconfig
@@ -1,5 +1,10 @@
 CONFIG_DEFAULT_HOSTNAME="localhost"
 CONFIG_SYSVIPC=y
+CONFIG_HUGETLBFS=y
 CONFIG_NO_HZ=y
 CONFIG_HIGH_RES_TIMERS=y
 CONFIG_PREEMPT_VOLUNTARY=y
```

配置Hugepage默认大小以及个数：

cmdline增加三个属性`default_hugepagesz=32M hugepagesz=32M hugepages=16`，配置Hugepage为32M大小，并且可使用个数为16。

```
+++ b/arch/arm64/boot/dts/rockchip/rk3588-linux.dtsi
@@ -6,7 +6,7 @@

 / {
        chosen: chosen {
-               bootargs = "earlycon=uart8250,mmio32,0xfeb50000 console=ttyFIQ0
irqchip.gicv3_pseudo_nmi=0 clk_gate.always_on=1 pm_domains.always_on=1
root=PARTUUID=614e0000-0000 rw rootwait";
+               bootargs = "earlycon=uart8250,mmio32,0xfeb50000 console=ttyFIQ0
irqchip.gicv3_pseudo_nmi=0 clk_gate.always_on=1 pm_domains.always_on=1
root=PARTUUID=614e0000-0000 rw rootwait default_hugepagesz=32M hugepagesz=32M hugepages=16 ";
        };

        cspmu: cspmu@fd10c000 {
```

重新编译内核烧写，开机后查看/proc/meminfo确认是否添加成功

```
[root@RK3588:/userdata]# cat /proc/meminfo | grep Huge
HugePages_Total:      16
HugePages_Free:       16
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:      32768 kB
Hugetlb:         1048576 kB
```

9.3 使用API接口获取user cmd区域或者ep info区域后，操作该区域导致内核报错或者报错Bus error

有两种原因会导致这个问题出现：

1. 使用memset或者memcpy等接口去操作这块内存。
2. 应用的编译优化超过了-O1。

第一种情况下检查代码去掉memset或者memcpy接口，使用for循环操作。 第二种情况可以降低优化等级，或者在编译参数中添加`-fno-tree-loop-optimize`关闭循环优化。

9.4 Windows驱动编译环境构建

当前驱动开发验证平台：**系统版本Windows10 2004，wdk版本10.0.19041.685**。

Windows开发目录结构及驱动架构说明请参考开发包清单章节。

9.4.1 环境搭建步骤

1. 确认系统版本

右键电脑属性，查看系统版本 或者 win+r 输入winver查看，主要关注操作系统内部版本：19042.1052。

```
版本                 Windows 10 专业版
版本号               20H2
安装日期             2021/6/5
操作系统内部版本     19042.1052
```

2. 下载安装vs2019以及WDK，参考网上教程即可。
3. Windows主机禁用驱动程序强制签名，这里提供3种方法：
   - 高级启动选项：

     重启计算机，在启动过程中进入高级启动选项（通常需要在启动时按F8键）。 在高级启动选项菜单中，如果看到”禁用驱动程序签名强制”或类似选项，并且这个选项被选中，那么数字签名校验被关闭。 使用命令行（仅用于临时禁用）：
   - 打开命令提示符（管理员）。 输入以下命令： bcdedit /set testsigning on
   - 使用组策略编辑器（适用于Windows Pro及更高版本）：

     打开”本地组策略编辑器”（gpedit.msc）。 导航到”计算机配置” -> “管理模板” -> “系统” -> “驱动程序安装”。 查看”设备：允许未签名的驱动程序安装”的设置。如果该设置被启用，则表示数字签名校验可能被禁用。

   重启计算机，如果在桌面的右下角看到”测试模式”字样，这表示数字签名校验被禁用。数字签名没有禁用则驱动会安装失败。
4. 加载工程编译

   vs工程文件在build/windows/rk_projects目录下，打开rk_projects.sln即可。

   解决方案包含三个项目：

项目	类型	配置	说明
rkep_driver	内核驱动	Debug/Release	生成rkep_driver.sys驱动文件
rk_cc	静态库	Debug/Release/Debug_Tiny/Release_Tiny	PCIe通信核心库，Tiny版本用于RK182x芯片
examples	GUI应用	Debug/Release	测试工具，支持固件下载、速率测试、视频测试

   编译成功后会在build/windows/rk_projects/install目录下输出文件：

```
install/
├── examples.exe              # 测试工具可执行文件
├── rk_cc/                    # 静态库及头文件
│   ├── include/              # API头文件
│   └── rk_cc.lib             # 静态库文件
└── rk_driver/                # 驱动文件
    ├── rkep_driver.pdb       # 驱动调试符号
    └── rkep_driver/          # 驱动目录
        └── rkep_driver.sys   # 驱动文件
```

9.5 Windows应用执行报错：MSVCP140.dll和vcruntime140.dll找不到

下载安装 Visual C++ Redistributable for Visual Studio 2015

9.6 Windows开启调试模式报错尝试修改调试程序设置时出错。该值受安全引导策略保护，无法进行修改或删除。

Windows10进入BIOS中，关闭安全启动。把“secure boot”设为“disable”

10. Chips Connect API参考

详细参考《Rockchip_Developer_Guide_PCIE_EP_Standard_Card_APIs》文档。